

Failure and Recovery in Transaction Management

What happens when the power fails mid-transaction, and how the recovery manager restores a correct database using nothing but a log.

Agenda

- 1 Types of transaction failure
- 2 System failure vs. media failure
- 3 The recovery manager's job
- 4 Log-based recovery, WAL, and a concrete violation to trace
- 5 Undo and redo, checkpointing, rollback and rollforward
- 6 A step-by-step crash-recovery trace: classify, redo, undo
- 7 Media-failure contrast, and key takeaways

Types of Transaction Failure

● Transaction Failure

Logical error: a constraint violation, a deadlock victim selection, a division by zero, or an explicit ROLLBACK. The DBMS and hardware are fine — only this one transaction needs undoing.

● System Crash

The DBMS, OS, or machine stops unexpectedly — but the disk remains intact. Everything in volatile memory at the moment of the crash is lost.

● Media (Disk) Failure

The storage device itself is damaged or destroyed. Unlike a system crash, the data ON DISK is no longer trustworthy or accessible at all.

System Failure vs. Media Failure

● System Failure

Volatile memory (RAM) is lost. Disk is intact.
Recovery replays the log against the existing disk data — no backup needed.

● Media Failure

The disk itself is lost or corrupted. Recovery must restore from the most recent backup, then apply the log to bring it forward to the point of failure.

A backup you have never restored is not a backup. Teams routinely discover, only during an actual media failure, that their backup files were silently incomplete or never being written. The only way to trust a backup is to periodically actually restore it — on a test system — and confirm the result is complete and consistent.

The Recovery Manager's Job

The recovery manager restores the database to a correct, consistent state after any failure — concretely, it must guarantee that Atomicity and Durability both hold despite the failure.

● Atomicity, guaranteed

Every transaction that had NOT committed must show none of its effects at all, exactly as if it had never run.

● Durability, guaranteed

Every transaction that HAD committed before the failure must still show its effects afterward.

To do this without trusting anything held only in volatile memory at the moment of a crash, the recovery manager relies on a log — a sequential, append-only record of every change, written to stable storage independently of the data files themselves.

Log-Based Recovery & Write-Ahead Logging

Each log record captures: the transaction ID, the data item changed, its old value (for undo), and its new value (for redo). The write-ahead logging (WAL) protocol states the rule that makes this reliable:

A log record describing a change must be written to stable storage BEFORE the data page it describes is written to disk.

1. Log record written

Old value, new value, transaction ID — flushed to stable storage.

2. Data page written

The actual table/index page is updated on disk — only after step 1.

If the data page were written first and the crash happened before the log record, there would be no record of what changed or how to undo it. Writing the log first guarantees the only possible failure mode is a change the redo step can fix.

What WAL Prevents: A Concrete Violation

Suppose transaction T5 writes item F (old value 500, new value 800). Now break the WAL rule on purpose: the DBMS flushes F's updated data page to disk first, and only afterward would write [T5, write F, old=500, new=800] to the log. The system crashes between those two steps.

On disk right now: F = 800. In the log: no record of this write exists at all.

1. Data page written first

F's page now reads 800 on disk, before any log record exists.

2. Crash before the log record

[T5, write F, old=500, new=800] never reaches disk. Nothing records the change.

The result is the worst case: a change reached disk with no record of its old value, so recovery cannot undo it or even know it happened. WAL's ordering rule (log first) rules this state out.

Tracing the Violation Through Two Outcomes

● Case A: T5 had not yet committed

F = 800 is on disk, but no log record exists to undo it with. T5 never committed, so Atomicity requires F to return to 500. Recovery cannot restore it, and cannot even detect that the write happened.

● Case B: T5 did commit

The commit is safe only by accident. Had the crash landed a moment earlier, the same disk state (F = 800, no log record) would exist for a transaction that never committed. The log alone cannot tell the two cases apart.

The lesson: WAL keeps the log a complete, trustworthy record, because each log record reaches stable storage before its data page. Skip that ordering and recovery loses the ability to tell committed writes from uncommitted ones.

Undo and Redo

Undo

Reverses the effect of a transaction that had NOT committed at the time of the crash, using each log record's old value to restore the data item. Every uncommitted transaction active at the moment of the crash must be fully undone — Atomicity demands it leaves no partial trace.

Redo

Reapplies the effect of a transaction that HAD committed, using each log record's new value, to guarantee its change is actually present on disk — WAL only guarantees the log record was written before the crash, not that the data page write had completed.

Redo is idempotent on purpose: reapplying an already-durable write causes no harm — it just writes the same new value again.

Checkpointing

Naively, recovery would mean replaying the ENTIRE log from the first transaction the database ever ran — growing more expensive without bound. Checkpointing solves this.

Flush all dirty pages to disk

Every change committed before this instant is now durably on disk.

Write CHECKPOINT record to the log

Marks the boundary: nothing before this point needs to be replayed.

On a later crash, recovery starts HERE

Not from the beginning of the log — only from the last checkpoint forward.

Why Start at the Checkpoint, Not the Beginning

● Without a checkpoint

Naive recovery replays every record, 1 through 13. That includes T1's START, its writes to A and B, and its COMMIT, even though T1 finished long ago and its changes are already on disk. Redoing finished work is wasted effort, and over months of history that waste is unbounded.

● With the checkpoint at record 5

Recovery provably needs only records 6 through 13: T2 (6-8), T3 (9-10), and T4 (11-13). Records 1 through 4 are skipped. The work is bounded by what happened since the last checkpoint, however long the database has been running.

The takeaway: recovery time is bounded by the time since the last checkpoint, not by the database's entire lifetime.

Rollback and Rollforward

● Rollback

Undoing a transaction's changes — either because it aborted during normal operation, or because recovery determined it was still uncommitted at crash time. Mechanically identical to “undo.”

● Rollforward

Reapplying committed changes from a log to bring a database forward to a target point in time. Mechanically identical to “redo” — the term used after a media failure: restore the last backup, then rollforward through the log.

Worked Walkthrough: The Log Up to the Crash

Four transactions run against Accounts (A–E). A checkpoint falls partway through; the crash comes last.

#	Log record	Note
1	[T1 START]	
2	[T1, write A, old=100, new=200]	
3	[T1, write B, old=50, new=150]	
4	[T1 COMMIT]	T1 fully committed
5	[CHECKPOINT]	Dirty pages flushed; disk matches log through #4
6	[T2 START]	
7	[T2, write C, old=10, new=20]	
8	[T2 COMMIT]	T2 fully committed, after the checkpoint
9	[T3 START]	
10	[T3, write D, old=5, new=99]	
11	[T4 START]	
12	[T4, write E, old=1, new=2]	
13	[T4 COMMIT]	T4 fully committed, after the checkpoint
—	CRASH	System failure — RAM lost, disk intact. T3 never committed.

Step 1: Where Recovery Starts

The checkpoint at record 5 tells recovery where to begin. Records 1-4 need no attention at all: T1 committed before the checkpoint, so its changes to A and B are already guaranteed safe on disk.

Records 1-4: skip

T1 START, its writes to A and B, and T1 COMMIT. All committed before the checkpoint, so there is nothing to redo or undo.

Record 5: CHECKPOINT

Disk matches the log through record 4. Analysis starts here and reads forward through record 13.

Every later step looks only at records 6 onward, plus the commit-or-not evidence for each transaction. Step 2 classifies them.

Step 2: Classifying Every Transaction

Each transaction active at or after the checkpoint is classified by where its COMMIT record falls relative to the crash point.

Transaction	Outcome by crash time	Action	Evidence in the log
T1	Committed before checkpoint	None needed	COMMIT at #4, before checkpoint #5: already durable
T2	Committed after checkpoint	REDO	[T2 COMMIT] #8 appears before the crash, so REDO
T3	Never committed	UNDO	No [T3 COMMIT] anywhere in the log before the crash, so UNDO
T4	Committed after checkpoint	REDO	[T4 COMMIT] #13 appears before the crash, so REDO

Redo: T2 and T4. Undo: T3. No action: T1. This classification is the entire input to the next two steps.

Step 3: Redoing the Committed Work

Every committed transaction found after the checkpoint is redone, using each record's new value.

● REDO C = 20

From record 7: [T2, write C, old=10, new=20]
T2 committed at record 8, after the checkpoint.
Set C to its new value, 20.

● REDO E = 2

From record 12: [T4, write E, old=1, new=2]
T4 committed at record 13, after the checkpoint.
Set E to its new value, 2.

Why redo is safe to repeat: setting C to 20 twice leaves C at 20. If the page write already reached disk, redo changes nothing; if it never did, redo supplies the missing change. Either way the result is the same. That is what idempotent means here.

Step 4: Undoing the Uncommitted Work

T3 never issued COMMIT, so none of its writes may survive. Undo walks backward through T3's own records and restores each old value.

- **UNDO D = 5**

From record 10: [T3, write D, old=5, new=99]
Restore D to its old value, 5: the value before T3 touched it. The new value 99 is discarded.

- **Why undo, not redo**

Having a log record for T3's write does not entitle it to survive. The record exists so the write can be reversed with certainty, not guessed at.

Undo and redo answer opposite questions: redo uses the new value to decide what must survive; undo uses the old value to decide what must vanish. Both are needed for Atomicity and Durability together.

The Result

Item	Final value	Why
A	200	From T1 — committed before the checkpoint, untouched
B	150	From T1 — committed before the checkpoint, untouched
C	20	From T2 — redone
D	5	From T3 — undone back to its original value
E	2	From T4 — redone

This is exactly the state the database would be in if T1, T2, and T4 had each run to completion and T3 had never run at all — precisely what Atomicity and Durability together require.

Rollback and Rollforward, Applied to This Example

● Rollback: T3

T3's undo is a rollback. Recovery determined that T3 was still uncommitted at crash time, so D goes from 99 back to its old value, 5.

● Rollforward: T2 and T4

T2's and T4's redo is a rollforward. Their committed changes are reapplied, setting C to 20 and E to 2, bringing the database forward to the moment of failure.

Same mechanics, same goal: A=200, B=150, C=20, D=5, E=2 is exactly the state a correct serial run would produce, with T3 never having run.

If This Had Been a Media Failure Instead

- **What happened: system crash**

RAM was lost, but the disk survived. The log is intact, so UNDO and REDO replay it against the existing data files. No backup is needed.

- **Had the disk itself been lost**

The data files and the log both lived on the destroyed disk, so no UNDO or REDO is possible. Restore the latest full backup, then rollforward through archived log records to reach the same end state.

Same numbers, different route: A=200, B=150, C=20, D=5, E=2 again, but only if the backup and archived log are intact. That is why an untested backup is the real risk on slide 4.

Key Takeaways

- **Failures split three ways:** transaction failure, system crash (RAM lost, disk intact), and media failure (disk gone). Each needs its own response.
- **The recovery manager** guarantees Atomicity and Durability using a log written independently of the data files.
- **WAL requires the log record first.** Breaking that ordering (slides 7-8) leaves uncommitted writes on disk with no way to undo them.
- **Undo and redo answer opposite questions:** undo reverses uncommitted work with old values; redo reapplies committed work with new values, and is safe to repeat.
- **The checkpoint bounds recovery** to the log since the last checkpoint, not the whole history.
- **The traced crash:** T1 needed no action; T2 (commit #8) and T4 (commit #13) were redone; T3, with no commit before the crash, was undone. Result: A=200, B=150, C=20, D=5, E=2.